

Lecture 2: Game Playing

Maciej Piasecki

Games: basic concepts

- Game – a sequence of actions led by the player (or players) in accordance with prescribed rules to reach a specific goal
- Game strategy – a complete set of rules that determines the player's move, chosen for game situations
- Formal definition of the game (von Neumann and Morgenstern) Game Theory:
„The game consists of a set of rules governing the possibilities of choice in a situation referred to as conflict of interest, in which each unit is trying to maximize its own profit and minimizing the profit of other entities (players). Game rules also define the amount of information available to each player and the number of winners and losers.”

Tasks of players

- Games require players
 - to make a series of decisions that increase their expected **payoff** (the amount of winnings)
 - and to take into account the factors determining the current situation (*game state*) and, usually, a limited time for actions
- Temporal restrictions do not normally allow for taking optimal decisions
- Undertaken decisions cannot be withdrawn
- The long-term outcome of the undertaken decisions is uncertain, because the decisions of the other players are unknown

Game types

- Number of players:
 - zero-player games, e.g., the game of life (John Conway's Game of Life)
 - one-player games, e.g., puzzle, solitaire
 - two-player games, e.g., chess, draughts (checkers), Othello, Go
 - multiplayer games, e.g., Halma, poker, and bridge
- Winning and losing conditions:
 - zero-sum games, where the win of one player is the loss of the second one (zero-sum games), e.g., chess, checkers, Othello, and Go
 - non-zero sum, e.g., prisoner dilemma
- Cooperation of players:
 - cooperative – players cooperate, e.g. bridge
 - non-cooperative – players do not cooperate
- Elements of randomness:
 - completely random game, e.g. roulette
 - partially random games, e.g., bridge and other card games
 - deterministic games, e.g., checkers, chess, Othello, and Go

Four Basic Game Categories Studied in AI

	Full information about a state	Incomplete information about the state
Determinism	chess, checkers	battleship (in Polish: gra w statki)
Randomness	backgammon	poker, bridge, scrabble

Logical games and AI

- Logical games — one of the oldest areas of AI applications
- Game rules are precisely defined with easily measurable results
- The history of AI is very strongly related to logical games
- 1946: Mauchley and Eckert built ENIAC, the first programmable computer
- 1948: N. Wiener considered chess as "representing a fundamental difference in the capabilities of the machine and the human mind"
- 1950: C. Shannon proposes the chess duel as the answer to the question of Turing
- Half a century later, the machine Deep Blue beat the current world champion (May 1997, the machine won 3,5:2:5). Did Deep Blue think?
- Years 1970. and 1980. brute-force approach

Development of game playing algorithms

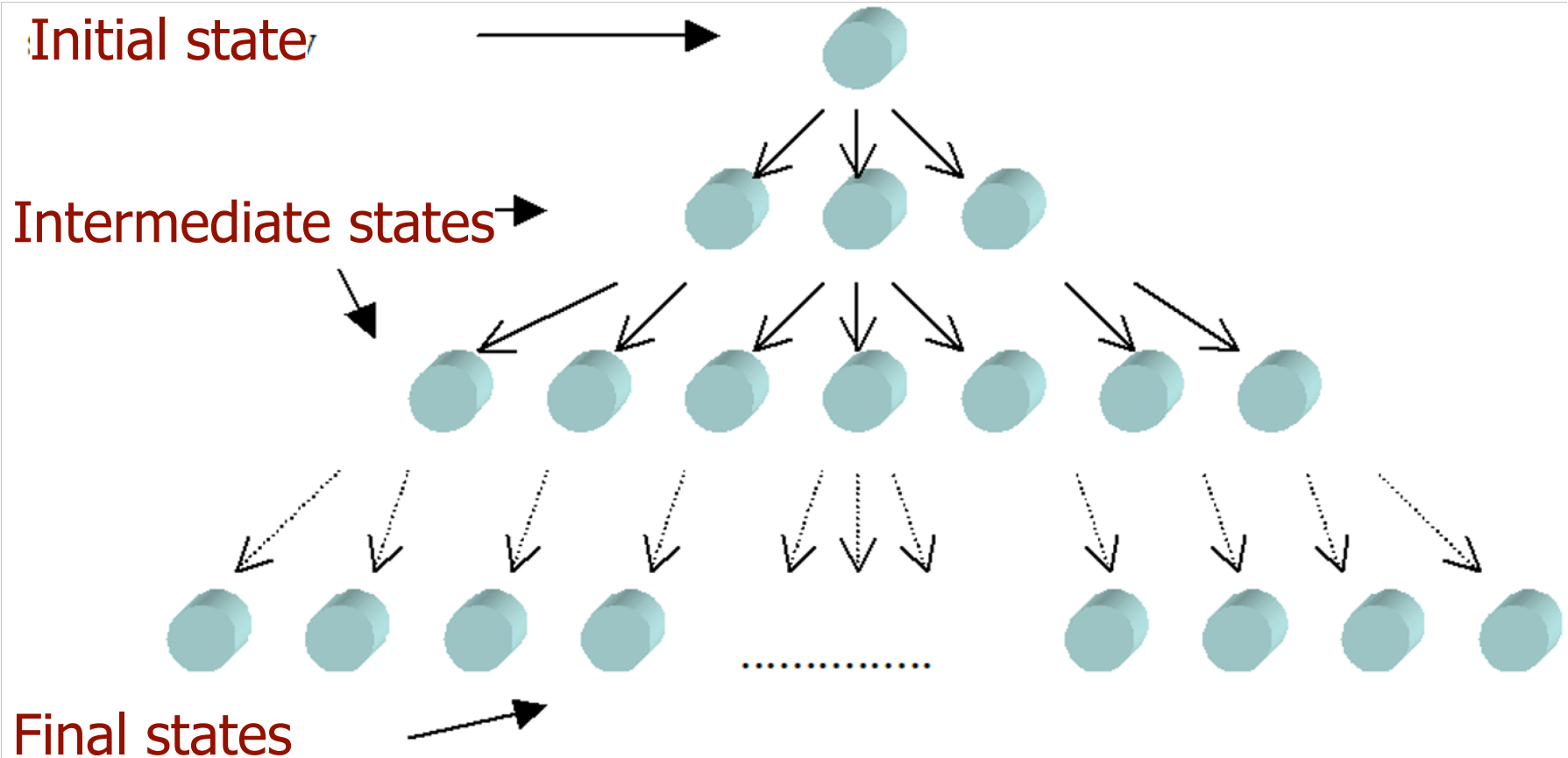
- The success of the Chess program series (Northwestern University) showed a strong correlation between the speed of the game tree searching and the quality of the game
- Late 1980s: a new phase – researchers developed many techniques for rapid search, including machine learning methods, championships of people and playing programs are held, computer games have become a fascinating task and a good testing ground for the verification of AI algorithms
- 1992: TD-Gammon was a computer backgammon program developed by Gerald Tesauro at IBM's Thomas J. Watson Research Center
- Chinook – the first computer champion in checkers on the board 8x8, won in 1994, mixed championship – people and computer programs
- 1997 – hot news spread that the chess winner was the computer Deep Blue (won against Garry Kasparow in 1997)
- Othello game – the best player is a computer program named Logistello
- Recently, Reinforcement Learning based programs beat humans in all games, e.g. Alpha Go in Go

Game tree

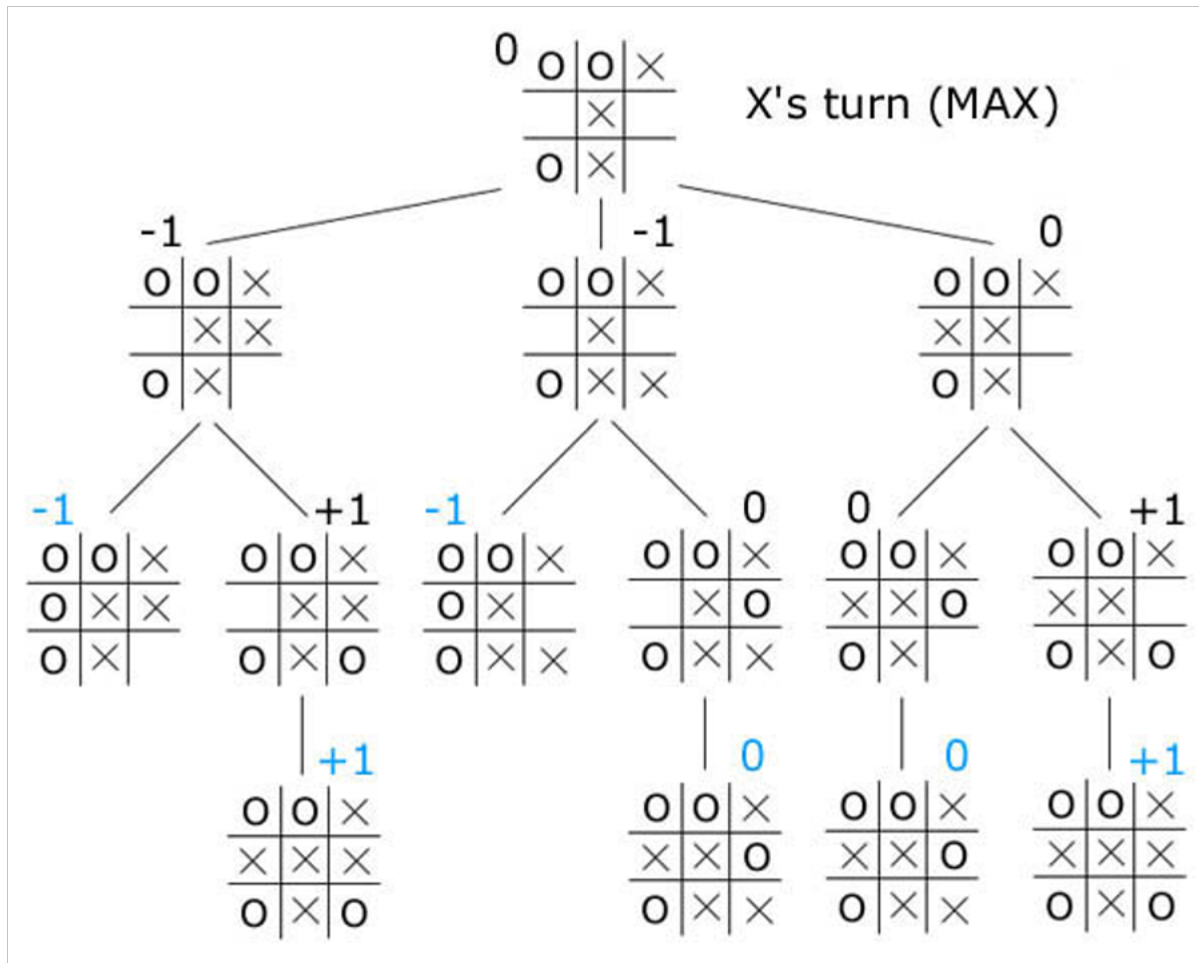
- In every step of the game, players perform their moves selected from the set of possible moves
- The game can be seen as a search for the player's movements that ensure his victory
- A tree can describe this process of searching
 - the **root** is the **initial state** of the game,
 - and the **nodes** — **states** resulting from the subsequent moves, possible at the given game state
- Even for simple games, the trees are very complicated, so different methods are used to assess the current state of the game and choose the next moves, without having to build the whole tree
- The quality of individual nodes, i.e., states of the game, is defined by a function that evaluates each node – a **heuristic evaluation function**

Game tree

- For most games, you can not examine the entire tree



Game tree example



- A game tree for tic-tac-toe
- Blue scores can be computed by looking at the current position – the payoff function
- To calculate the scores for the other positions, one must look ahead a few moves

<https://www.ocf.berkeley.edu/~yosenl/extras/alphabeta/alphabeta.html>

Evaluation Function (Heuristic Function)

- The game tree can not be developed until the end — we are forced to stop this process at some time
- 'The stopping time' generally results from the resources allocated for the process, for example, the time left for the decision
- The computer must make a move based on the partial game tree – i.e. knowledge about the states on the board achievable from a given state
- A human can assess the situation on the board and decide which of the players is in a good position, but a computer is not
- Therefore, the concept of **heuristic function**, called the **evaluation function**, was introduced
- In games, the heuristic value describes the “goodness” of the board situation(the game state) as a number, possibly the estimated probability of achieving victory for that state.
- Often, the evaluation function (heuristic) is a linear combination of the features characterising the game state:

$$f = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n$$

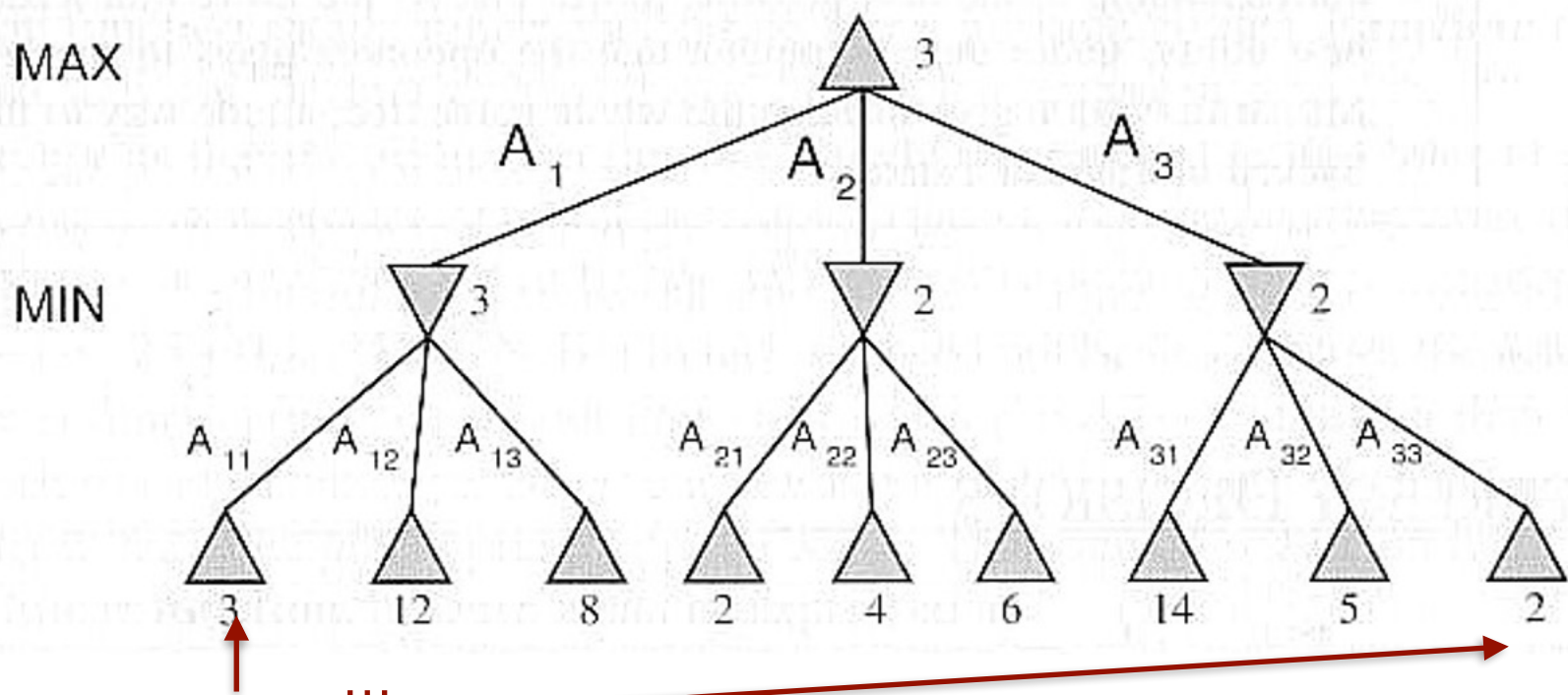
Brute-force (non-)solutions

- Blind search — searching through the game tree without a heuristic function — “brute force”
- All nodes of a game tree accessible from a given position are expanded and checked
- Concept of game tree — the analogy of solving problems by humans
- A human, while considering hypothetical scenarios
 - does not take into account all possible events
 - eliminates from consideration unlikely events, or those assessed at the beginning as not leading to the goal
- Very large scale of the problem,
 - e.g. for chess 35^{100} search tree nodes with 10^{40} different possible situations on the board

Min-Max Algorithm

- **Evaluation function** (**heuristic function**) — additional knowledge derived from the problem
 - compatible with the final state
 - should be positively correlated with 'the chances for winning'
- Unknown exact value of the evaluation function
 - discovering it is a challenge
- Min-max idea: choosing movements leading to the higher-scoring states
- During the game:
 - the opponent tries to choose actions leading to his victory
 - important assumption: **his topmost game-playing skills**
 - **⇒ he selects the worst moves for us (Min!)**
- Always looking at the problem from the perspective of 'our' player:
 - one should choose those operators that lead to the states evaluated as the best ones (**Max**)
 - his opponent will select operators leading to conditions that minimise the value of the evaluation function (**Min**)
- **⇒ Min-Max Algorithm**

Minmax Tree



Payoff values

(Russel & Norvig 1995, pp. 125)

- Scored in a bottom-up way from the perspective of the player

Min-Max Algorithm

```
minimax(player,board)
```

```
  if(game over in current board position)
```

```
    return winner and payoff value
```

```
  children = all legal moves for player from this board
```

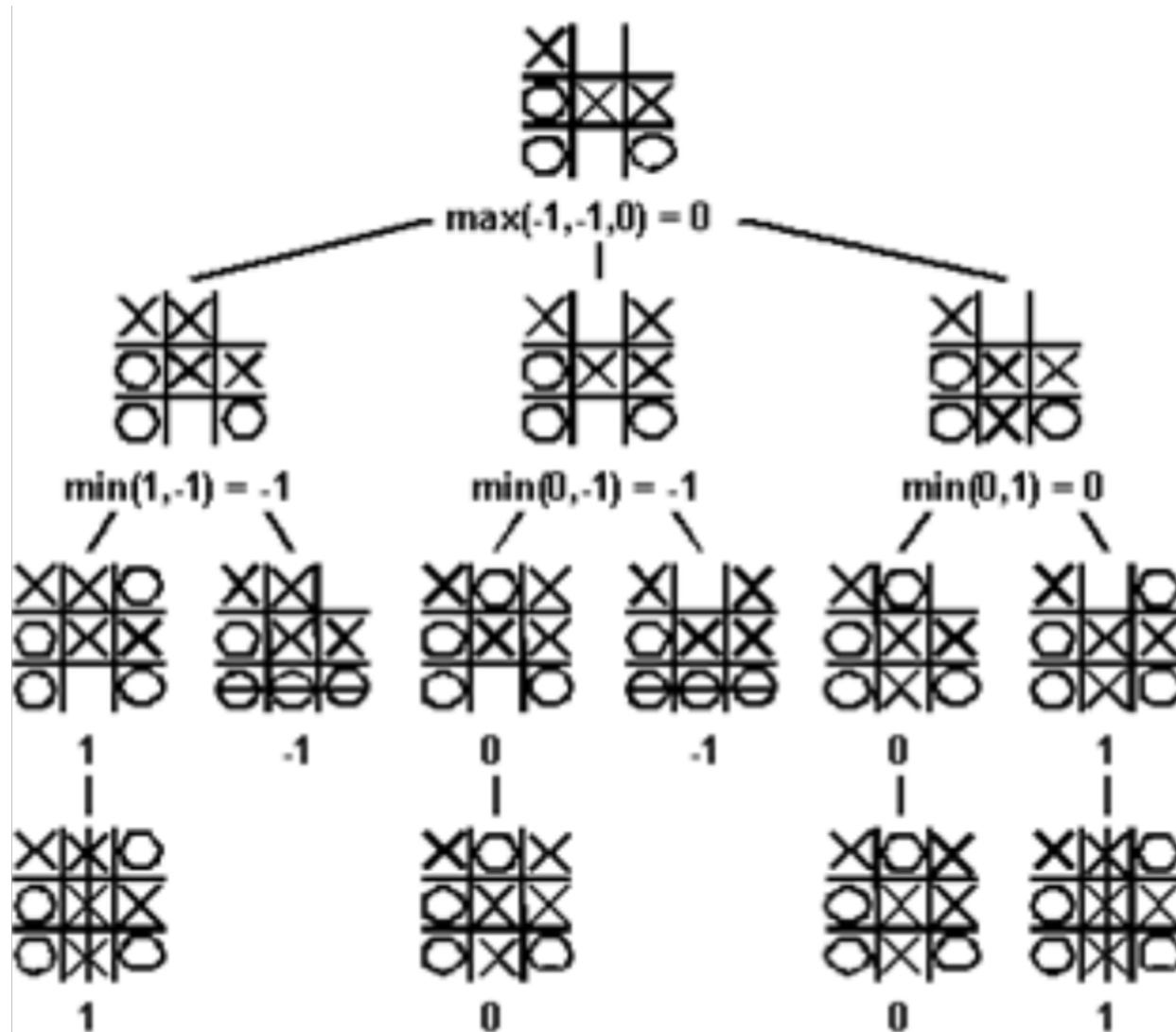
```
  if(max's turn)
```

```
    return maximal score of calling minimax on all the children
```

```
  else (min's turn)
```

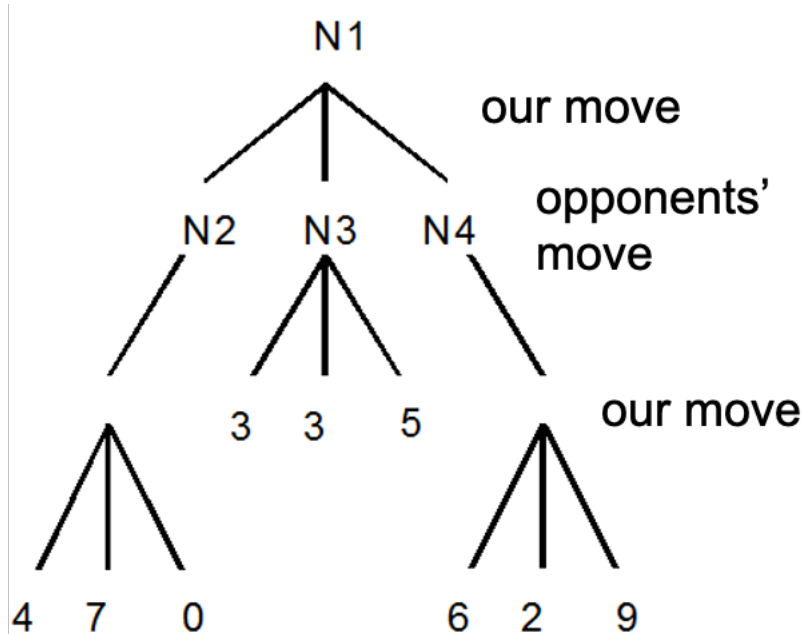
```
    return minimal score of calling minimax on all the children
```

Min-Max Algorithm



<http://www.cs.cmu.edu/~adamchik/15-121/lectures/Game%20Trees/Game%20Trees.html>

Min-Max Algorithm



- Evaluation function values
- Provided for the leaves,
- Are next propagated up the tree, up to the root,
- Player move - MAX
- Oponent move - MIN
- Thus,
- $N2 = \min(4, 7, 0) = 0;$
- $N3 = \min(3, 3, 5) = 3;$
- $N4 = \min(6, 2, 9) = 2;$
- $N1 = \max(N2, N3, N4) = \max(0, 3, 2) = 3$

Min-Max Algorithm: Cut-off Point

○ Predefined **constant depth**

- a sudden change of situation below the cut-off depth
 - * e.g. Checkers: an uncrowned piece (*man*) reaches the farthest row forward (king row) $\Rightarrow$ becomes a *king*
- 'unpredictable' changes of the evaluation function

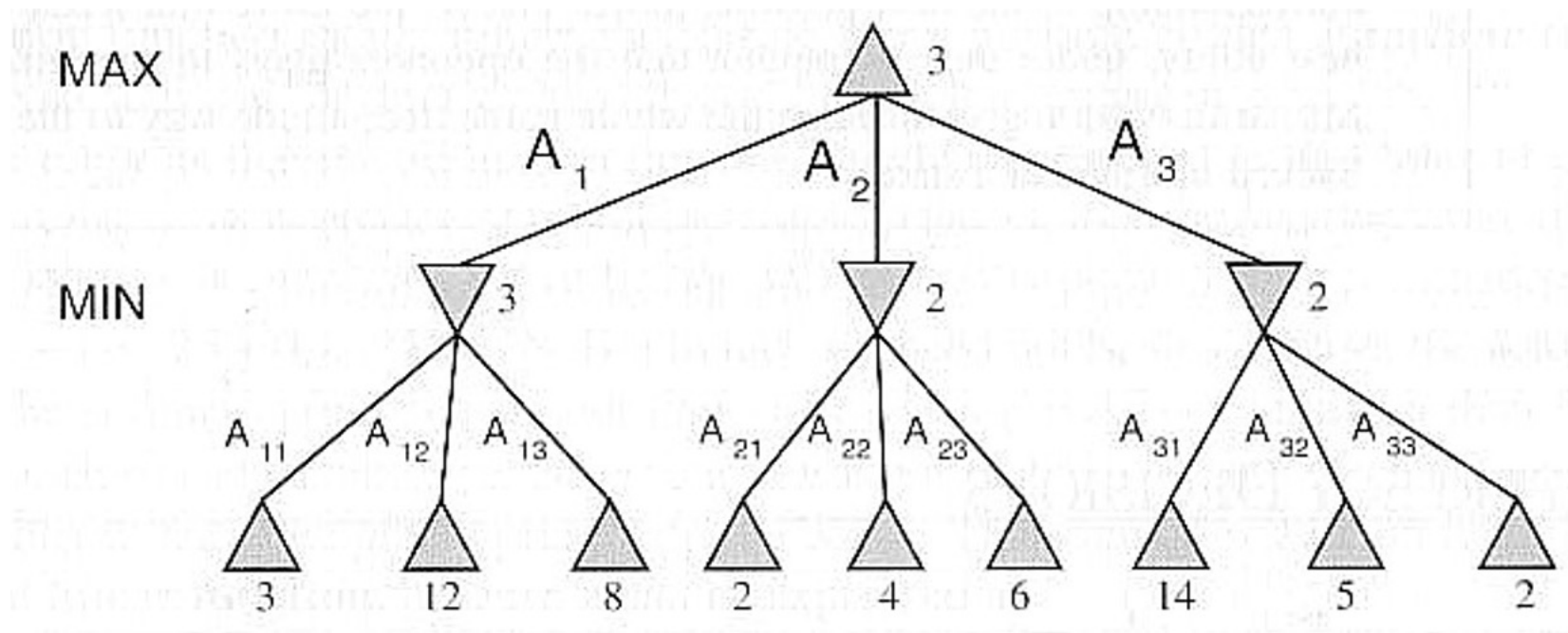
○ **Quiescent positions**

- in which the game state does not express a possibility of a drastic change of the evaluation in the 'near future'
- **quiescence search**
 - * additional search (e.g. below the maximal default depth)
 - * in order to achieve a quiescent position

○ **Horizon problem**

- an opponent's move that changes very significantly (drastically) the game state and evaluation

Modified Search – Discovering ‘Promising’ Nodes



Alpha-beta Pruning – improvement over Minimax:

- ❑ the middle subtree – the value 2 in the leaf causes that we can **omit** the rest of the subtree as $2 < 3$
- ❑ different situation in the right subtree – **disadvantageous node ordering!**

Alpha-beta pruning idea – omission of those subtrees that do not change the score values of the nodes on the upper level

$\alpha\beta$ Pruning Algorithm

```
function ALPHA-BETA-SEARCH(state) returns an action
   $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$ 
  return the action in ACTIONS(state) with value  $v$ 
function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for each  $a$  in ACTIONS(state) do
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s,a), \alpha, \beta))$ 
    if  $v \geq \beta$  then return  $v$ 
     $\alpha \leftarrow \text{MAX}(\alpha, v)$ 
  return  $v$ 
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow +\infty$ 
  for each  $a$  in ACTIONS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s,a), \alpha, \beta))$ 
    if  $v \leq \alpha$  then return  $v$ 
     $\beta \leftarrow \text{MIN}(\beta, v)$ 
  return  $v$ 
```

(Russel & Norvig 2003, str. 170)

See also: <https://www.youtube.com/watch?v=xBXHtz4Gbdo>

$\alpha\beta$ Pruning Algorithm

○ Properties

□ complexity

- * Strongly dependent on **the order** in which the nodes are expanded
- * Optimistic time complexity — $O(b^{d/2})$, effective branching factor = square root of b
- * E.g. for chess 6 instead of 35
- * Pessimistic (Knuth & Moore, 1975): $O((b/\log b)^d)$

○ Node expansion order problem

□ additional heuristics needed

□ e.g.

- * Take actions (of pieces, pawns, stones, etc.)
- * Dangers
- * Forward moves
- * Backward moves

□ a part of the h function from the perspective of the A^* scheme

Constructing good evaluation function

- For instance: game Othello (Reversi)
- Board and rules
- Exemplary features, e.g.,
 - the number of pieces (discs) of both players
 - the number of legal moves of both players
 - number of opponent pieces adjacent to the own pieces
 - the number of the opponent pieces adjacent to the empty field, etc.
 - differences in numbers for both opponents
- $E = w_1 \cdot F_1 + w_2 \cdot F_2 + \dots + w_n \cdot F_n$
- Optimisation
 - heuristic (intuition) — try and error
 - Genetic Algorithm: games between artificial players
 - statistical learning
 - Reinforcement Learning

	a	b	c	d	e	f	g	h
1								
2								
3								
4				○	●			
5				●	○			
6								
7								
8								

Deep Blue® – the chess machine

○ Deep Blue ® — IBM (Campbell et al., 2002)

- the first chess machine “that defeated then-reigning World Chess Champion Garry Kasparov in a six-game match in 1997” (Deep Blue II)
- a computer system: dedicated hardware and complex software
- developed since the mid-1990s
- Deep Blue — the first version (Deep Blue I) lost to Kasparov in 1996

○ Main features

- “a single-chip chess search engine,
- a massively parallel system with multiple levels of parallelism,
- a strong emphasis on search extensions,
- a complex evaluation function,
- and effective use of a Grandmaster game database”

Deep Blue® – decision making

○ Large searching capacity

- originating from α - β Min-Max scheme
- highly non-uniform and highly selective search
- parallel, two level searching architecture
 - * higher level software search
 - * lower level hardware search — chess processors
 - * move generation
 - * limited depth search

○ Evaluation procedure

- complex, beyond just a function
- Opening book
 - * 4000 positions manually described (Grandmaster Joel Benjamin, et al.)
- Extended book
 - * derived from Grandmaster game database
 - * statistical analysis of positions in 700,000 game database

Deep Blue® – dual search

Software search

Host processor, C code

Explores tree near root

No quiescence search

Complex recursive extensions

Transposition table

Uses hardware search as dynamic evaluation function

Flexible

Hardware search

Chess chip, state machines

Explores tree near leaves

Complex quiescence search

Mostly local extensions

No transposition table

Uses on-chip static evaluation function

Hardwired, limited configurability

Deep Blue – the chess chip (processor)

○ Move generation

- an 8×8 array of combinatorial logic — a hardwired finite state machine
- generates only one move at a time — from all possible ones
- "move ordering, preferably as close to best-first as possible"
 - * "first generating captures (ordered from low-valued pieces capturing high-valued pieces to high-valued capturing low-valued), followed by non- capture moves (ordered by centrality)."

○ Evaluation function

- fast evaluation
 - * mostly "basic piece values with square-based location adjustments"
- and slow evaluation
 - * "chess concepts such as square control, pins, X-rays, king safety, pawn structure, passed pawns, ray control, outposts, pawn majority, rook on the 7th, blockade, restraint, color complex, trapped pieces, development, and so on."

○ Search control

- 4-5 ply with quiescence (midgame), deeper in end game
- "a fixed-depth null-window search, which includes a quiescence search"
- a repetition detector
 - * "a 32-entry circular buffer of the last 32 ply."
 - * "the numbers of pieces displaced in each of the last 32 positions"

Deep Blue – The dual credit algorithm.

General α - β Min-Max scheme

- "a depth-limited version of alpha-beta using the negamax formulation"

Extend forcing/forced pairs of moves

- expectation dependent
- fractional extensions
- delayed extensions — based on accumulated credit

Dual credit

- "to separate and accumulate the credit for the two sides separately"

Preserve the search envelope

- "to avoid an oscillating search"

```

1 int DC(
2     position p,
3     int alpha, int beta,
4     int depthToGo,
5*    float myCredit, float hisCredit)
6 {
7     int numOfSuccessors;
8     int bestScore;
9     int i;
10    int sc;
11*   float newCredit;
12*   int extensionAmount;
13
14*   if (hisCredit >= CREDIT_LIMIT) {
15*       extensionAmount = ceiling(hisCredit - CREDIT_LIMIT);
16*       hisCredit = hisCredit - extensionAmount;
17*       myCredit = max(myCredit - extensionAmount, 0);
18*       depthToGo = depthToGo + extensionAmount;
19*   }
20
21   if (depthToGo == 0) { return Evaluate(p); }
22
23   bestScore = alpha;
24   numOfSuccessors = GenerateSuccessors(p);
25   for (i = 1; i <= numOfSuccessors; i++) {
26*       sc = -DC(p.succ[i], -beta, -alpha, depthToGo - 1,
27*               hisCredit, myCredit);
28*       if (sc > bestScore) {
29*           newCredit = GenerateCredit();
30*           if (newCredit > 0)
31*               sc = -DC(p.succ[i], -beta, -alpha, depthToGo - 1,
32*                       hisCredit, myCredit + newCredit);
33*           if (sc > bestScore)
34*               bestScore = sc;
35*       }
36*       if (bestScore >= beta) { return bestScore; }
37*   }
38   return bestScore;
39 }
```

Deep Blue – Evaluation function

- A sum of feature values
- Chess chip
 - 8000 different “patterns”, and each is assigned a value
 - correspond to different chess situations
 - registers and weights
- Evaluation function generator
 - static and dynamic feature values
 - run only at the root of the search tree

Deep Blue evaluation registers – examples

Function	Number of registers	Data bits
Rooks on seventh rank	12	8
Bias	1	8
Opposing rook behind passed	1	9
Mpin and hung	1	7
Pinned and simple hung	1	8
Hung	4	7
Xraying	2	6
Pinned and hung	1	7
Permanent pin and simple hung	1	8
Knight trap	6	8
Rook trap	8	8
Queen trap	2	8
Wasted pawns	2	6
Bishop pair	2	7
Separated passed	2	8
Missing wing	2	10
Bishops of opposite colors	2	6
Evaluation control	2	32
Side to move	2	4

Deep Blue evaluation registers – examples

Function	Number of tables	Table entries	Data bits
Multiple pawns	2	80	8
Minor on weak	2	192	12
Self block	2	320	5
Opponent block	2	128	4
Back block	2	160	5
Pinned	2	128	8
Mobility	8	128	9
Pawn structure	2	160	32
Passed pawns	2	320	32
Joint signature	1	256	8
Rooks on files	2	192	10
Bishops	4	128	11
Pawn storm	2	128	18
Pawn shelter	2	384	14
Development	1	256	9
Trapped bishop	1	128	8
Signature	2	128	20
Contempt	1	256	8
Piece placement	1	1024	10

Additional literature

- Campbell, Murray (1999). "Knowledge discovery in deep blue". Communications of the ACM. Association for Computer Machinery. 42 (11): 65–67. doi:10.1145/319382.319396
- Campbell, Murray; Hoane, A. Joseph Jr.; Hsu, Feng-hsiung (2002). "Deep Blue". Artificial Intelligence. Elsevier. 134 (1–2): 57–83. doi:10.1016/S0004-3702(01)00129-1. ISSN 0004-3702.